

messieR: Function Reference

Missingness, Errors-based Sample Size Inference for ESM Research

Karthik Tumuluru
KU Leuven

August 2026 (single-file research release)

Status of this manual

This document is the interim reference for the single-file release (`messieR_core.R`). The core depends only on base R and `stats`. For sizeable runs using this package, `parallel` is recommended to install.

Contents

1	Conventions that apply across the API	2
2	Simulation	2
2.1	<code>sim_var1</code>	2
2.2	<code>is_stationary</code>	3
2.3	<code>resample_pool</code>	3
2.4	<code>default_block_length, default_stationary_p</code>	4
3	Missingness injection	4
3.1	<code>inject_mcar</code>	4
3.2	<code>inject_mar</code>	4
4	Estimation	4
4.1	<code>fit_var1</code>	4
5	Bootstrap inference	5
5.1	<code>boot_var1_all</code>	5
5.2	<code>boot_residual</code>	6
5.3	<code>boot_stationary</code>	6

5.4	<code>boot_moving_block</code>	6
5.5	Internal helpers	6
6	Monte Carlo error utilities	7
7	A worked path from real data to a prospective simulation	7

1 Conventions that apply

Gap-aware transitions. All fitting is built on transitions rather than observations. A transition enters a regression only when both of its endpoints are observed *and adjacent in time*: the series is lagged first and incomplete pairs are dropped second (the transition-level analogue of listwise deletion). Deleting rows before lagging would splice non-adjacent observations into false one-step transitions and attenuate the estimated dynamics; the core never does this. In other words, the machinery can only see gaps that are *coded*. If an archive records unanswered prompts as absent rows rather than NA rows, reindex the series onto the prompt schedule (inserting NA rows, plus one NA row at each day-boundary) before calling anything here.

Pattern reimposition. The residual bootstrap rebuilds each bootstrap series from a fully observed starting row and then reimposes the observed missingness mask, so every bootstrap refit faces the same gap structure the data did.

Two inferential conventions. Interval columns (`ci_lo`, `ci_hi`, and the simulation-study indicator `covered`) come from the studentized (percentile- t) bootstrap interval. The `pval` and `reject_05` columns come from a percentile bootstrap p -value. These are related but distinct constructions; for a badly calibrated resampler they can disagree, and they should be reported separately rather than treated as complements of one another.

Coefficient naming. For a bivariate system, coefficients are labelled `phi11`, `phi12`, `phi21`, `phi22`, where `phiIJ` is the effect of variable J at time $t-1$ on variable I at time t (row = equation, column = predictor).

Reproducibility. Nothing in the core sets seeds. The analysis scripts seed *per replicate*, so grid results are bit-identical across platforms and across interrupted, resumed runs.

2 Simulation

2.1 `sim_var1`

Simulate a stationary VAR(1) series under a chosen innovation source.

Usage

```
sim_var1(n_obs, phi,
         delta = NULL, sigma = NULL,
         residual_pool = NULL, innov_fn = NULL,
         resample = c("iid", "block"), block_size = NULL,
```

```
burn_in = 1000L)
```

Arguments

- **n_obs**: number of retained observations T .
- **phi**: $k \times k$ transition matrix; must satisfy `is_stationary(phi)`.
- **delta**: intercept vector (default zero).
- **sigma**: innovation covariance for the Gaussian source.
- **residual_pool**: a matrix of empirical residuals to draw innovations from (e.g. `fit_var1(Y)$residuals` of a real participant).
- **innov_fn**: a function `f(n)` returning an $n \times k$ matrix of innovations (used for skew-normal or Student- t conditions).
- **resample, block_size**: how the pool is resampled when **residual_pool** is supplied; "block" preserves short-range dependence, with **block_size** defaulting to `default_block_length(n_obs)`.
- **burn_in**: observations generated and discarded before the retained window (default 1000), which leaves the retained series at the stationary distribution to numerical accuracy.

Innovation precedence. **residual_pool** beats **innov_fn** beats **sigma**; if none is supplied the call stops. This ordering is what lets the same simulator serve parametric stress tests and fully empirical prospective simulations without a flag.

Value. A $n_{\text{obs}} \times k$ numeric matrix.

2.2 is_stationary

Usage `is_stationary(phi, tol = 1e-8)`

Returns TRUE when the spectral radius of **phi** is below $1 - \text{tol}$. Used as a guard by **sim_var1** and as a screen when fitting real participants.

2.3 resample_pool

Draw rows from a residual pool, independently or in contiguous blocks.

Usage

```
resample_pool(pool, T_target, method = c("iid", "block"), block_size = NULL)
```

Arguments

- **pool**: <matrix> Rows to draw from, typically `fit_var1(Y)$residuals`.
- **T_target**: <integer> Number of rows to return.

- **method:** <character> "iid" draws rows independently; "block" draws contiguous runs, wrapping circularly, which preserves short-range dependence in the pool.
- **block_size:** <integer> Block length for `method = "block"`; defaults to `default_block_length(T_target)`.

Details

Called internally by `sim_var1()` when `residual_pool` is supplied. Exposed because it is useful on its own when writing a custom innovation generator.

Value

A $T_{\text{target}} \times k$ <matrix>.

2.4 default_block_length & default_stationary_p

Data-driven tuning defaults for the two block procedures.

Usage

```
default_block_length(T_use)
```

```
default_stationary_p(T_use)
```

Arguments

- **T_use:** <integer> Series length.

Details

`default_block_length` returns $\lceil 5.03 T^{1/4} \rceil$, the rule of Jentsch and Lunsford, used as the fixed block length for the moving-block bootstrap. `default_stationary_p` returns $1/\lceil 2.5 T^{1/4} \rceil$, a Politis–White style restart probability, so the expected block length under the stationary bootstrap is roughly half the fixed length used by the moving-block procedure. Both grow only as $T^{1/4}$, which is why the block-junction artefacts documented in the accompanying thesis recede slowly with series length.

Value

A <numeric> vector of length 1.

Examples

```
default_block_length(100)      # expected block length at T = 100
default_stationary_p(100)     # restart probability; expected block length 8 at T = 100
```

3 Missingness injection

3.1 inject_mcar

Usage `inject_mcar(Y, rate = 0.30, col = 2L)`

Sets each row of column `col` to NA independently with probability `rate`. Missing completely at random: the mechanism ignores the data.

3.2 inject_mar

Usage

```
inject_mar(Y, rate = 0.30, coef = 0.5, col = 2L, predictor = 1L)
```

Missing at random with an *observed* driver: the probability that column `col` is missing at time t is a logistic function of column `predictor` at time $t - 1$ (standardised), with slope `coef`; the intercept is solved numerically so the marginal missingness rate equals `rate`. Because the driver is a lagged *regressor* of the fitted model, this mechanism is ignorable for the transition-level analysis: in the thesis simulations it thinned the sample without tilting it. It does *not* represent state-dependent (MNAR) non-response, and results under it should not be read as reassurance about that case.

Value (both injectors). The input matrix with NAs inserted.

4 Estimation

4.1 fit_var1

Gap-aware equation-by-equation OLS for a VAR(1).

Usage `fit_var1(Y, min_obs = 10L, full = TRUE)`

Arguments

- `Y`: $T \times k$ series; NAs permitted anywhere.
- `min_obs`: minimum usable transitions; below it, returns NULL rather than a fragile fit.
- `full`: when FALSE, returns only `phi_hat` and `se` (the bootstrap refit path, roughly halving grid runtime).

Details. Builds transition pairs from the original series and keeps only pairs whose two endpoints are observed and adjacent (Section ??). Both equations share one design matrix $[1, Y_{t-1}]$, so a single cross-product inverse serves both; this is the optimisation that took the production grid from days to hours.

Value (`full = TRUE`). List: `phi_hat` ($k \times k$), `intercept`, `Sigma_hat`, `residuals` (one row per usable transition), `se`, `pvals` (t tests per coefficient).

5 Bootstrap inference

5.1 boot_var1_all

One call, all procedures: the recommended entry point.

Usage

```
boot_var1_all(Y, B = 999L, alpha = 0.05, true_phi = NULL,  
              include_moving_block = FALSE)
```

Arguments

- **Y**: the observed series (NAs permitted).
- **B**: bootstrap replicates per procedure.
- **alpha**: interval level (0.05 gives 95% intervals).
- **true_phi**: simulation studies only; when supplied, adds **covered** (studentized interval contains the truth) and bias columns.
- **include_moving_block**: adds the fixed-block variant alongside the stationary bootstrap (off by default; it was not run in the thesis production grid).

Value. A long data frame, one row per procedure per coefficient. Column dictionary:

- **procedure**: `wald`, `residual_iid`, `stationary` (and `moving_block` when requested).
- **coef**: `phi11 ... phi22`.
- **est**: the OLS point estimate (identical across procedures).
- **est_bc**: `<numeric>` Bootstrap bias-corrected estimate, $2\hat{\phi} - \overline{\hat{\phi}^*}$. Present for the three bootstrap rows; NA for the Wald row, which has no resampling distribution to correct against. NA for Wald).
- **se**: the analytic standard error. Populated for the Wald row only; NA for the bootstrap rows, whose uncertainty is carried by the interval rather than by a single scalar.
- **ci_lo**, **ci_hi**: studentized (percentile-*t*) interval for bootstrap rows; the normal-quantile interval for the Wald row.
- **pval**: percentile bootstrap *p*-value for bootstrap rows; the *t*-test *p*-value for the Wald row.
- **reject_05**: `pval < .05` as an integer.
- **covered**: `<integer>` 1 when the interval contains **true_phi**, 0 otherwise; NA if **true_phi** is omitted.
- **bias**, **bc_bias**: **est** and **est_bc** minus the truth; NA unless **true_phi** is supplied.

Every row of the returned frame repeats the same **est**, because all procedures share one point estimate and differ only in how uncertainty around it is constructed.

Mind the two-conventions note in Section ?? when reading **covered** against **reject_05**.

5.2 `boot_residual`, `boot_stationary` & `boot_moving_block`

The three resampling procedures called by `boot_var1_all()`. Use them directly when only one procedure is wanted.

Usage

```
boot_residual(Y, B = 999L, alpha = 0.05)
```

```
boot_stationary(Y, B = 999L, alpha = 0.05, p = NULL)
```

```
boot_moving_block(Y, B = 999L, alpha = 0.05, block_size = NULL)
```

Arguments

- **Y**: `<matrix|data.frame>` The observed series (NAs permitted).
- **B**: `<integer>` Bootstrap replicates.
- **alpha**: `<numeric>` Interval level.
- **p**: `<numeric>` (stationary only) Restart probability; defaults to `default_stationary_p(nrow(Y))`.
- **block_size**: `<integer>` (moving block only) Fixed block length; defaults to `default_block_length(nrow(Y))`.

Details

`boot_residual` is model-based. It fits the VAR, resamples the residual rows independently, rebuilds each series through the estimated recursion starting from the first *fully observed* row, reimposes the observed missingness mask, and refits. Starting from a complete row matters under missingness: seeding the recursion with an NA propagates it through the entire bootstrap series. With complete data the behaviour is unchanged.

`boot_stationary` and `boot_moving_block` are nonparametric. Both resample the *observed series* in blocks and impose no VAR structure, differing only in whether the block length is geometric or fixed. Both create an artificial junction wherever two non-adjacent observations are concatenated. In the accompanying thesis this produced coverage between 0.79 and 0.92 against a nominal 0.95 across all series lengths studied, for both variants, so they are documented as evaluated comparators rather than as recommendations. The residual-block bootstrap of Brüggemann, Jentsch and Trenkler, which resamples residuals in blocks and so never splices the observed series, is the untested middle case.

Value

A `<list>` with `phi_hat`, `phi_bc` (bias-corrected estimate), `se_boot`, the interval bounds, and the percentile bootstrap *p*-values. `boot_var1_all()` assembles these into the long data frame documented above.

See Also

```
boot_var1_all(), default_block_length()
```

Examples

```

phi_mat <- matrix(c(0.5, 0.1, 0.1, 0.5), nrow = 2)
sim_data <- sim_var1(n_obs = 100, phi = phi_mat, sigma = diag(2))

# Model-based residual bootstrap only
br <- boot_residual(sim_data, B = 99)
br$phi_hat

```

5.3 Internal helpers

`boot_engine` (drives the resample-refit loop for any series constructor) and `boot_summary` (assembles the per-coefficient output rows) are internal; their arguments may change when the file becomes a package.

6 Monte Carlo error utilities

Usage

```

mcse_proportion(x, n_sim = sum(!is.na(x)))
mcse_mean(estimates, n_sim = sum(!is.na(estimates)))
mcse_bias(estimates, true_value = NULL, n_sim = sum(!is.na(estimates)))
mcse_mse(estimates, true_value, n_sim = sum(!is.na(estimates)))

```

Binomial MCSE for proportions (coverage, Type I error, power), $sd/\sqrt{n}$ for means and bias, and the delta-style MCSE for MSE. Reference values at $n_{\text{sim}} = 1000$: a proportion near 0.5 carries an MCSE of about 0.016; near 0.2, about 0.013; a coverage near 0.95, about 0.007. Report these next to every simulated quantity (ADEMP).

7 A worked path from real data to a prospective simulation

```

fit1 <- fit_var1(Y_participant)          # any real participant
Ysim <- sim_var1(100, fit1$phi_hat,
               delta = fit1$intercept,
               residual_pool = fit1$residuals)
boot_var1_all(inject_mar(Ysim), B = 999)

```

Three lines take an empirical participant's dynamics and innovation distribution, generate a new study of any planned length under a realistic observation process, and return calibrated inference for it. That loop is the intended use of the codebase.